

Project Checkpoint 3: Evaluator

3/27/2026

80 Points Possible

 Add Comment

3/2/2026 to 3/30/2026

Details

In this assignment, you will implement the evaluator for our language. This is the final step needed to have a "working" programming language which evaluates the parsed AST. The majority of this assignment is ensuring evaluation following the correct evaluation order, control flow, and scoping.

Setup/Submission

(<https://ufl.instructure.com/courses/555493/files/103969638/download>)

- Setup: [PlcProject \(Evaluator Patch\).zip](https://ufl.instructure.com/courses/555493/files/105093415?wrap=1) (<https://ufl.instructure.com/courses/555493/files/105093415?wrap=1>)  (https://ufl.instructure.com/courses/555493/files/105093415/download?download_frd=1) contains a **patch** of new/updated source files. These files should be merged in with your *existing* project, *replacing* any existing files (the files being replaced are part of our provided API structure and should not be being modified). Copying the `src` folder into the root project should match the path structure properly.
- Submission: You will submit a zip of **only** the `src` folder, which includes both sources and tests. Using `Right Click src` + `Compress to Zip` should have the expected behavior, or alternatively `zip -r submission.zip src` from the command line. The name of the file is irrelevant.
 - The first test submission is **Thursday, March 12: Project Checkpoint 3: Evaluator (TEST 1 - Core)** (https://ufl.instructure.com/courses/555493/assignments/6905756?module_item_id=12536195)
 - Only "core" rules are graded (see spec).
 - The second test submission is **Tuesday, March 24: Project Checkpoint 3: Evaluator (TEST 2 - All)** (https://ufl.instructure.com/courses/555493/assignments/6905757?module_item_id=12536196)
 - All rules are graded. As a TEST 2 submission, this cannot be dropped!
 - The final submission is **Friday, March 27.**

Evaluator Overview

The interpreter is responsible for evaluating code - the AST. To do this, will use the visitor pattern where each `visit` method evaluates the corresponding `Ast` class, returning the appropriate value (if any). Our interpreter is going to be dynamic, and so we will need to take into account the possibility for failures like undefined variables or invalid types.

For example, evaluating the AST represented by `1 + 2 * 3` returns the runtime value of the number `7`. The wording here is important - the runtime value of `7` is a `RuntimeValue.Primitive` containing a `BigInteger` as opposed to just an `int` (or similar). Values in the specification will generally be represented as unwrapped values, so `NIL` means `new RuntimeValue.Primitive(null)`.

Managing Scope

The majority of this assignment is building up an understanding of scopes. Scopes are used to track what functions/variables an identifier refers to. The same name can be used in different scopes (such as a class field and

function parameter), and thus scopes are represented using a hierarchy to allow this. The implementation of the `Scope` class is provided, but you need to have an understanding of how it works to use it properly.

For example, in Java we could have code which looks like this:

```
int x = 0;
if (true) {
    x = 1;
    int y = 2;
}
```

After evaluating this, `x` is either `0` or `1` and `y` is... `2`? Actually, after this if statement `y` isn't anything - it's simply undefined - and this is because it was defined within the scope of the if statement which has finished. On the other hand, the value of `x` has indeed changed, since it is not defined in the scope of the if statement but is accessible from it. This process of restoring the scope when exiting the block is important (especially when invoking functions/methods) and should happen regardless of any exceptions being thrown.

Crafting Interpreters

Chapters 7-11 (starting with [Evaluating Expressions \(https://craftinginterpreters.com/evaluating-expressions.html\)](https://craftinginterpreters.com/evaluating-expressions.html) [\(https://craftinginterpreters.com/evaluating-expressions.html\)](https://craftinginterpreters.com/evaluating-expressions.html)) cover the different ideas needed for the evaluation process. The most important piece to understand is [Scoping \(https://craftinginterpreters.com/statements-and-state.html#scope\)](https://craftinginterpreters.com/statements-and-state.html#scope), which has a wide variety of behavior and can be quite tricky when it comes to functions and especially methods. I highly recommend reading these to help with your understanding of the evaluation process and this assignment.

Evaluation Semantics

There are two critical concepts that must be understood in the evaluator:

- **Evaluation Order** refers to the order used to evaluate code throughout the program. Unless otherwise stated, we will use **top-to-bottom, left-to-right** evaluation order with **eager validation** (i.e., validating any semantic conditions immediately after the necessary values have been evaluated prior to evaluating any subsequent values). For example, `true - undefined` errors at `true` because a boolean cannot be used in subtraction, while `true + undefined` errors at `undefined` because the plus operation is also used for string concatenation which allows a boolean so long as the other value is a string.
 - Recall you can (and should) use `log` for testing whether expressions are evaluated.
- **Scoping** determines the variables/functions that are accessible during evaluation. The spec will be explicit when scopes are introduced, but you must determine when the scope exits and ensure the Evaluator's scope is restored to the correct value. This restoration should be performed **even if an exception is thrown** (consider: why?).
 - Note you can experiment with JavaScript which will generally have consistent scoping behavior, but always cross-check observed behavior with the specification itself.

If an error should be thrown, you must use the provided `EvaluationException` class and should include the most relevant `Ast` at the point of failure if accessible (i.e., the most-specific AST relevant to the condition being checked). We will not rely heavily on the `Ast`, but it can be useful for debugging and helps represent a clear point of failure. The `String message` is simply a descriptive message for your own debugging and will not be used in tests.

Environment Specification

The `Environment` class contains a scope with definitions for "native" variables/functions, which effectively act like the standard library of our language. Most of these functions add "power" to our language by implementing functionality that couldn't be accomplished in any other way - for example, printing to `System.out`. Additionally, these allow having access to

variables/functions that can be easily used in testing - it's generally better for automated tests to be a bit more isolated, but these help with manual testing too so we'll live with it.

Variable/Function	Description	Examples
<code>sqrt(number)</code>	Returns the <code>sqrt</code> of an <code>Integer</code> / <code>Decimal</code> (see <code>BigInteger</code> / <code>BigDecimal</code> methods). For <code>BigDecimal</code> , use a <code>MathContext</code> applicable to doubles (hint: how many bits?) and <code>HALF_EVEN</code> rounding like division - check JavaDocs! Note: The resulting behavior for precision is often unintuitive.	<pre>sqrt(4.0); //2 sqrt(4.1); //2.0248...</pre>
<code>range(start, end)</code>	Returns a <code>List</code> of integers from <code>start</code> (inclusive) to <code>end</code> (exclusive). The arguments to <code>start</code> / <code>end</code> must both be integers in our language (aka <code>BigInteger</code> s) and satisfy <code>start <= end</code> .	<pre>FOR elem IN range(1, 5) DO log(elem); END //1, 2, 3, 4</pre>

Ast Types

The following table lists all AST types, a description of behavior beyond the provided grammar, and example inputs. As above, **pay particular attention to errors** and how potentially-ambiguous inputs should be parsed.

The recommend implementation path is to work from the bottom up as in the Parser, however note that the Evaluator has significantly more overlap (e.g. `log(...)` is desirable for testing evaluation order in binary expressions, which requires functions).

1. Start with `Ast.Expr.Function` following guidance in lecture to implement `log`, which is critical for testing evaluation order (this is provided).
2. Then implement remaining expressions: `Variable` / `Group` are both straightforward, `Binary` is complex but self-contained, and lastly `Property` / `Method` which involves receiver scoping. You can choose to save prototypal inheritance for later as it relates heavily to `ObjectExpr`.
3. Then implement statements: `Let` / `If` are both approachable starting points, `Assignment` for variables is important for testing scope, and `Def` / `Return` are the most difficult. Pay particular attention to scoping and control flow, especially with functions.
4. Finally, implement `ObjectExpr` which requires defining fields/methods, building on `Let` / `Def`, and any remaining behavior related to prototypal inheritance.

AST Type	Description	Examples
<code>Source</code> (provided, excl. RETURN)	Evaluates a full program, with the following behavior: • Evaluate all statements, handling the possibility of <code>RETURN</code> being called from outside of a function (see <code>Stmt.Return</code> spec). • Return the value of the last evaluated statement.	<pre>log(1); log(2); log(3);</pre>
<code>Stmt.Let</code> (core)	Defines a variable in the current scope, with the following behavior: • The variable name must not be already defined in the <i>current</i> scope. • The value of the variable defaults to <code>NIL</code> if not explicitly given. • Return the value of the defined variable.	<pre>LET name; LET name = "value";</pre>

Stmt.Def	Defines a function in the current scope, with the following behavior: - The function name must not be already defined in the <i>current</i> scope. - The function implementation should have the following behavior: - The implementation follows the semantics of <i>static scoping</i>. - Within a new scope, define all parameters of the function to the corresponding values of the passed arguments. Ensure parameters are unique and that the number of arguments matches (hint: review our semantics around evaluation order). - Within <i>another</i> new scope (to allow shadowing), evaluate all <code>body</code> statements. - Return the value resulting from <code>RETURN</code> (how?), or else <code>NIL</code>. - Return the value of the defined function.	<pre>DEF name() DO log(1); END DEF name(param) DO log(param); END</pre>
Stmt.If (core)	Evaluates an <code>IF</code> statement, with the following behavior: - The <code>condition</code> must be a <code>Boolean</code>. - Within a new scope, evaluate all <code>thenBody</code> / <code>elseBody</code> statements (corresponding to the <code>TRUE</code> / <code>FALSE</code> condition). - Return the value of the last evaluated statement, or else <code>NIL</code>.	<pre>IF TRUE DO log("true"); END IF FALSE DO log(1); ELSE log(2); EN D</pre>
Stmt.For	Evaluates a <code>FOR</code> loop, with the following behavior: - The expression must be a primitive <code>Iterable</code>. - Iterating over the expression value <i>once</i>, and: - Within a new scope (to ensure closures maintain the value of that iteration), define the variable <code>name</code> with the value of the element (which has the <i>invariant</i> (hint, hint) that it is a <code>RuntimeValue</code>). - Within <i>another</i> new scope (to allow shadowing), evaluate all <code>body</code> statements. - Return <code>NIL</code>.	<pre>FOR e IN list(1, 2, 3) DO log(e); E ND</pre>

Stmt.Return	Evaluates a RETURN statement, with the following behavior: - Throws an exception to return the evaluated value, or else NIL. Hint: See "Return Statements" (https://craftinginterpreters.com/functions.html#return-statements) in Crafting Interpreters - you might need to define a new class! (and don't modify existing ones...). - Note: For simplicity, this will not validate that return is only used within functions (instead, handle this in Source - this means that testing stmt directly may leave return unhandled, but that's fine here).	<pre>DEF name() DO RETURN "value"; END name();</pre>
Stmt.Expression (provided)	Evaluates an expression statement, with the following behavior: Return the value of the evaluated expression.	<pre>variable; //okay! function();</pre>
Stmt.Assignment (variables core)	Evaluates an assignment statement, with the following behavior: - If the receiver is an Ast.Expr.Variable, the corresponding variable must be defined (see Ast.Expr.Variable spec) and it will be assigned the new value in the defining scope. - If the receiver is an Ast.Expr.Property, the corresponding property must be defined (see Ast.Expr.Property spec) and it will be assigned the new value in the <i>object's</i> scope (i.e. assignment should <i>never</i> mutate the prototype object itself, only the current object's properties). - Otherwise, the receiver is invalid and an error should be thrown. - Return the newly assigned value.	<pre>variable = value; object.property = value;</pre>
Expr.Literal (provided)	Evaluates a literal expression, returning a RuntimeValue.Primitive.	<pre>1 "string"</pre>
Expr.Group (core)	Evaluates a group expression, returning the wrapped expression.	<pre>(expr)</pre>
Expr.Binary (core)	Evaluates a binary expression, with the following behavior: +: If <i>either</i> operand is a String, the result is their concatenation using RuntimeValue.print(). Else, the left operand must be a BigInteger / BigDecimal, the right operand must be the <i>same type as the left operand</i>, and the result is their addition. - / *: The left operand must be a BigInteger / BigDecimal, the right operand must be the <i>same type as the left operand</i>, and the result is their subtraction/multiplication. /: The left operand must be a BigInteger / BigDecimal, the right operand must be the <i>same type as the left operand</i>, and the result depends on the type/values:	<pre>1 + 2 1 * 2 1 + 2 * 3 // 1 + (2 * 3) 1 * 2 + 3 // (1 * 2) + 3</pre>

	◦ <code>BigInteger</code> will use floor division, like regular integers. ◦ <code>BigDecimal</code> will require a rounding mode, which will be <code>HALF_EVEN</code>. This rounds midpoints to the nearest even value (<code>1.5</code>, <code>2.5</code> -> <code>2.0</code>). ▪ Note: The resulting behavior for precision is often unintuitive. ◦ In both cases, divide by zero must be handled and throw our error. • <code>== / !=</code> : Tests for equality using <code>Objects.equals</code> (note this is not the standard equals method - check the Javadocs for details). • <code>< / <= / > / >=</code> : The left operand must be <code>Comparable</code>, and the right operand must be <i>same type as the left operand</i>. The result is <code>TRUE / FALSE</code> based on the specific operator, which you will have to interpret using <code>Comparable</code> (check the Javadocs). • <code>AND / OR</code> : Both operands (if evaluated) must be a <code>Boolean</code>. <code>AND/OR</code> have their standard logical meanings and use short-circuiting, as in Java.	
<code>Expr.Variable</code> (core)	Evaluates a variable, with the following behavior: • The variable <code>name</code> must be in scope. • Return the value of the defined variable.	variable
<code>Expr.Property</code>	Evaluates a property, with the following behavior: • The receiver must be an object (in our language). • The object must define the property <code>name</code>, following the rules of prototypal inheritance (i.e. in it's own scope or recursively inherited from a <code>prototype</code> object). • Return the value of the defined property.	object.property
<code>Expr.Function</code> (core)	Evaluates a function, with the following behavior: • The function <code>name</code> must be a <code>RuntimeValue.Function</code> in scope. • Return the result of invoking the function with the given arguments.	function() function(1, 2, 3)
<code>Expr.Method</code>	Evaluates a method, with the following behavior: • The receiver must be an object (in our language). • The object must define the function <code>name</code>, following the rules of prototypal inheritance (i.e. in it's own scope or recursively inherited from a <code>prototype</code> object). • Return the result of invoking the function with the object followed by the given arguments.	object.method() object.method(1, 2, 3)
<code>Expr.ObjectExpr</code>	Creates and returns an <code>ObjectValue</code> , with the following behavior: • The object <code>name</code> is the same as the AST's. • The object <code>fields</code> are defined in the object's scope, following the same requirements as <code>LET</code> in all other	OBJECT DO LET field; END OBJECT DO DEF method() DO END END

behavior.

- The object `methods` are defined as functions in the object's scope, following the same requirements as `DEF` in all other behavior except:
 - The function will take an additional parameter, passed first, of the method receiver (aka object) matching the calling convention established in `Expr.Method`. It will be defined to the variable `this`, (thus `this` cannot be the name of a given parameter).
 - Note that `this.field` is the only way to access fields within the method; `field` on it's own is not equivalent.

Tip: It is not recommended to try reusing `LET` / `DEF` for this - just copy the relevant behavior and adjust accordingly using tests to verify semantics.

Changelog

3/4	• Updated provided code (PlcProject (Evaluator Patch).zip (https://ufl.instructure.com/courses/555493/files/105093415?wrap=1)_ ↓ (https://ufl.instructure.com/courses/555493/files/105093415/download?download_frd=1)) with multiple corrections: ◦ <code>Main</code>: Fixed incorrect references to AdminLexer/AdminParser (part of lecture demos). ◦ <code>Evaluator</code>: Added <code>throws EvaluateException</code> to provided <code>Environment</code> functions. ◦ <code>EvaluatorTests</code>: Fixed incorrect return value for <code>sqrt</code>. ◦ <code>EvaluatorTests</code>: Fixed missing log <code>List.of()</code> argument for <code>range</code>. ◦ <code>EvaluatorTests</code>: Fixed <code>Op+ Evaluation Order Left Validation Order</code> behavior being incorrect for <code>+</code>. ◦ <code>EvaluatorTests</code>: Removed unnecessary (& incorrect) test for <code>range</code> in <code>testFunctionExpr</code>. • Clarified expected precision/rounding behavior for <code>Decimal</code> <code>sqrt</code>.
3/4	• Updated provided code (PlcProject (Evaluator Patch).zip (https://ufl.instructure.com/courses/555493/files/105093415?wrap=1)_ ↓ (https://ufl.instructure.com/courses/555493/files/105093415/download?download_frd=1)) to correct AdminEvaluator reference in <code>EvaluatorTests</code>.
3/23	• Removed bonus for concatenation of function/object values; this no longer applies after project changes.



(<https://ufl.instructure.com/courses/555493/modules/items/12536192>)



(<https://ufl.instructure.com/courses/555493/modules/>)